CYBERSECURITY LABORATORY SERIES

Cloning and Installing OSINT Tools from GitHub

A Beginner's Step-by-Step Manual

From git clone to a running tool, using three real repositories

web-check • GhostTrack • theHarvester

Joshua Moses

CyberTrace Foundation

August 2026 Edition

How to Use This Manual

This manual teaches one workflow, three times. The workflow is: use Git to clone a repository, install the dependencies the tool needs, and run it. Learn it once and you can install almost any tool on GitHub.

Parts 1 and 2 teach the workflow in general, with no particular tool in mind. Parts 3, 4, and 5 then apply it to three real OSINT tools, each a little different so you meet the common variations: a Node.js web application, a simple Python script, and a modern Python project that uses the newer uv installer.

Every major procedure follows the same three beats:

Beat	What it does
Explain	States what the procedure accomplishes and which choices matter, so you can recover if a screen looks different
Perform	Numbered steps with the exact commands to type
Verify	A specific check that proves the step worked before you move on

Each walkthrough ends with a **Troubleshooting** table covering the problems that actually come up. Commands to type appear in monospaced boxes. Buttons and on-screen items appear in **bold**.

Who This Is For

You should be comfortable installing ordinary applications and opening a terminal. You do not need any prior experience with Git, the command line, Python virtual environments, or Node.js. Every term is explained before it is used, and Appendix C collects the definitions in one place.

Before You Begin: Safe and Lawful Use

> **WARNING**
> All three tools gather information, and two can reach out to a target directly. Only use them against systems, domains, or identifiers you own or are explicitly authorized to investigate. These are instruments for security research and authorized assessment, not for surveilling or harassing anyone. Appendix D covers this in full, and it is worth reading before you run anything.

Contents

Part 1: Using Git to Clone a Repository

Before you can install any of the three tools in this manual, you need a copy of its code on your own machine. The tool that does this is Git, and the action is called cloning. This part explains what that means and walks you through it once, generically, so that the three tool-specific walkthroughs later can move quickly.

1. What Git and a Repository Actually Are

1.1 Terms Before We Use Them

Term	What it means
Git	A version control program that lives on your computer. It tracks changes to files and can copy whole projects from the internet. Git is the tool; GitHub is a website that hosts projects Git can talk to.
GitHub	A website that stores millions of Git projects. When you visit github.com/someone/something, you are looking at a project hosted there.
Repository (repo)	A single project, with all its files and its full history of changes. Each of the three tools in this manual is one repository.
Clone	To download a complete copy of a repository, including its history, onto your own machine, using Git. A clone stays linked to the original so you can pull updates later.
Commit	One saved change in a project's history. You will not make commits in this manual, but you will see the word.
Branch	A parallel line of development. Most projects have a main line called main or master, which is what you get by default.

1.2 Clone Versus Download ZIP

GitHub offers a green **Code** button with a "Download ZIP" option, and you might wonder why this manual uses Git at all. Two reasons.

- **Updating.** A cloned repository can be updated with a single command, git pull, which fetches only what changed. A ZIP file is a dead snapshot. To update it you download the whole thing again and lose any local changes.
- **Correctness.** Some projects read their own Git history at build time, or expect files that a ZIP export leaves out. Cloning gives you exactly what the developer intended.

For a security tool you will want to keep current, cloning is the right habit. Learn it once and it applies to every repository you will ever use.

2. Installing Git

2.1 Explain

Git is not always present by default. macOS and most Linux systems either include it or install it with one command. Windows needs a download. Check first, install only if needed.

2.2 Perform: Check Whether Git Is Already Installed

Open a terminal (on Windows, open **PowerShell** or **Command Prompt**; on macOS, open **Terminal**; on Linux, open your terminal application) and run:

```
git --version
```

If you see something like git version 2.45.2, Git is installed and you can skip to Section 3. If you see "command not found" or "not recognized", install it using the section for your system below.

2.3 Perform: Install Git on Windows

1. Open a browser and go to the official Git site:

   ```
   https://git-scm.com/download/win
   ```

2. The download for the 64-bit Windows installer starts automatically. Run the downloaded .exe.
3. Accept the defaults through the installer. The defaults are sensible for a beginner. If asked about a default editor, choosing anything you recognize is fine.
4. When the installer finishes, close every open terminal window, then open a new one so it picks up the new Git.

2.4 Perform: Install Git on macOS

The simplest route triggers Apple's own installer:

```
git --version
```

If Git is missing, macOS pops up a dialog offering to install the Command Line Developer Tools. Click Install and wait. When it finishes, run git --version again to confirm.

If you use Homebrew, you can instead run:

```
brew install git
```

2.5 Perform: Install Git on Linux

```
# Fedora
sudo dnf install -y git

# Ubuntu / Debian
sudo apt update && sudo apt install -y git
```

2.6 Verify

> **VERIFY**

> git --version prints a version number on its own line, with no error.
>
> The command works in a newly opened terminal, not just the one the installer left open.

2.7 One-Time Setup (Optional but Recommended)

Git likes to know who you are. This is only required if you ever commit changes, which this manual does not, but setting it now avoids a warning later:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

3. Cloning a Repository: The General Procedure

3.1 Explain

Every clone in this manual follows the same three moves: pick where the copy will live, run git clone with the repository's URL, then step into the new folder. Learn this once here and the tool-specific parts become short.

3.2 Perform

1. Decide where your projects will live and create a folder for them. Keeping tools in one place saves confusion later.

   ```
   # Windows (PowerShell)
   mkdir $HOME\tools
   cd $HOME\tools

   # macOS / Linux
   mkdir -p ~/tools
   cd ~/tools
   ```

2. Get the repository's clone URL from GitHub. On the repository's page, click the green **Code** button and copy the HTTPS URL. It always ends in .git, for example https://github.com/owner/project.git.

3. Run the clone command with that URL:

   ```
   git clone https://github.com/owner/project.git
   ```

   Git creates a new folder named after the project, downloads every file and the full history into it, and reports its progress.

4. Step into the newly created folder:

   ```
   cd project
   ```

5. Confirm you are in the right place by listing the files:

   ```
   # Windows
   dir

   # macOS / Linux
   ```

```
ls -la
```

[SCREENSHOT REQUIRED]

Capture: A terminal immediately after a successful git clone

Show: The git clone command, its progress output ending in "done", and the new project folder listed by ls or dir

Caption: Figure 3-1: A completed clone, with the project folder created and ready

3.3 Verify

VERIFY

Git reported "Receiving objects: 100%" and then "done".

A new folder named after the project now exists.

Listing the folder shows the project's files, including a README.md and usually a file describing dependencies.

3.4 What You Just Downloaded

Take a moment inside a freshly cloned folder before installing anything. Two files matter almost every time:

- **README.md** is the front door. It tells you what the tool is, what it needs, and how to install and run it. Read it first, every time. The install steps in this manual come from each project's own README, verified against the live repository.
- **A dependency file** tells the installer what other software the project needs. Its name depends on the language: requirements.txt or pyproject.toml for Python, package.json for Node.js. Part 2 explains these.

TIP

Reading the README before running commands is the single habit that separates a smooth install from an hour of confusion. The three tools here each declare their needs slightly differently, and the README is where they say so.

3.5 Troubleshooting the Clone

Symptom	Likely cause	Fix
git: command not found	Git is not installed, or the terminal predates the install	Complete Section 2, then open a new terminal
fatal: repository not found	A typo in the URL, or the repo is private	Recopy the HTTPS URL from the green Code button on the repo page
Could not resolve host: github.com	No internet, or a proxy or firewall is blocking	Check your connection; on a managed network, ask about proxy settings
destination path already	You cloned here before	Delete or rename the existing folder, or cd

Symptom	Likely cause	Fix
exists		into it and run git pull instead
Clone is extremely slow	A very large repository, or a slow link	Let it finish; for huge repos, git clone --depth 1 <url> fetches only the latest snapshot

Part 2: Installing a Project's Dependencies

4. What Dependencies Are and Why They Exist

4.1 Explain

Almost no program is written entirely from scratch. Developers build on top of existing libraries, which are bundles of code that solve common problems so nobody has to reinvent them. The libraries a project needs in order to run are its **dependencies**.

When you clone a repository, you get the project's own code but not its dependencies. Those have to be installed separately, which is what this part is about. The good news is that projects declare exactly what they need in a standard file, and a single command reads that file and installs everything listed.

4.2 The Two Ecosystems in This Manual

The three tools split across the two most common ecosystems you will meet. Knowing which one a project uses tells you which commands to run.

	Python	Node.js
Used by	GhostTrack, theHarvester	web-check
Dependency file	requirements.txt or pyproject.toml	package.json
Package installer	pip (or the newer uv)	npm or yarn
Language runtime you install first	Python 3	Node.js
Typical install command	pip install -r requirements.txt	yarn install or npm install

5. Python Dependencies and Virtual Environments

5.1 Explain

Two of the tools are Python programs. Before installing their dependencies, you need to understand one concept that saves an enormous amount of grief: the virtual environment.

Your system has one shared Python installation. If every project dumped its dependencies into that shared space, two projects that need different versions of the same library would collide, and you could break system tools that quietly rely on Python. A **virtual environment** is a private, isolated Python workspace for a single project. Libraries you install inside it stay inside it and cannot disturb anything else.

> **WARNING**
> Modern Linux distributions, including Fedora, now actively block installing packages into the system Python and will show an "externally-managed-environment" error if you try. This is deliberate. The correct answer is not to force it; the correct answer is to use a virtual environment, exactly as this

section describes.

5.2 Perform: Confirm Python Is Present

```
python3 --version
pip3 --version
```

You want Python 3.12 or newer for theHarvester specifically. If either command is missing, install Python 3 first: sudo dnf install -y python3 python3-pip on Fedora, sudo apt install -y python3 python3-pip python3-venv on Ubuntu, or from https://www.python.org/downloads/ on Windows (tick "Add Python to PATH" during the Windows installer).

5.3 Perform: Create and Activate a Virtual Environment

1. Inside the project folder you cloned, create a virtual environment named venv:

   ```
   python3 -m venv venv
   ```

   This makes a venv subfolder holding a private copy of Python. You only do this once per project.

2. Activate it. The command differs by system:

   ```
   # macOS / Linux
   source venv/bin/activate

   # Windows (PowerShell)
   venv\Scripts\Activate.ps1

   # Windows (Command Prompt)
   venv\Scripts\activate.bat
   ```

3. Confirm it worked. Your prompt now starts with (venv). That prefix is how you know you are working inside the isolated environment.

NOTE

When you are done working with a tool, type deactivate to leave the environment. The next time you use the tool, cd back into its folder and run the activate command again. You do not recreate the environment; you just reactivate it.

5.4 Perform: Install the Dependencies

With the environment active, install whatever the project's dependency file lists. The -r flag means "read the requirements from this file":

```
pip install -r requirements.txt
```

Some projects list their dependencies in pyproject.toml instead. For those, pip install . (note the dot, meaning "this folder") reads that file and installs the project along with its dependencies.

5.5 Verify

VERIFY

> pip finishes with a line like "Successfully installed" followed by a list of package names and versions.
>
> There are no red ERROR lines. Yellow WARNING lines about pip versions are harmless.
>
> pip list shows the packages the project asked for.

5.6 A Note on the Newer uv Installer

Some projects, including theHarvester, now recommend a newer tool called **uv** in place of the venv plus pip sequence. uv does the same two jobs, creating an isolated environment and installing dependencies, but faster and in one step. You do not need it for GhostTrack, and Part 5 walks through it in full for theHarvester. The concept is identical; only the commands are shorter.

6. Node.js Dependencies

6.1 Explain

One tool, web-check, is a Node.js project. Node.js is a runtime that lets JavaScript run outside a browser, and it comes with its own dependency system. A Node project declares its dependencies in package.json, and an installer reads that file and downloads everything into a folder called node_modules.

There are two installers in common use, npm and yarn. npm ships with Node.js itself. yarn is an alternative that some projects, including web-check, prefer. When a project ships a yarn.lock file, that is a strong signal it expects yarn.

6.2 Perform: Install Node.js

1. Download the LTS (Long Term Support) version from the official site:

   ```
   https://nodejs.org/en/download
   ```

   web-check needs Node.js version 18.16.1 or newer. The current LTS comfortably exceeds that. On Fedora you can instead run sudo dnf install -y nodejs, and on Ubuntu sudo apt install -y nodejs npm, though the versions in distribution repositories can lag; the official installer is safest.

2. Confirm both Node and its bundled npm are present:

   ```
   node --version
   npm --version
   ```

3. Install yarn, which web-check uses. The modern way is through Node's built-in Corepack:

   ```
   corepack enable
   yarn --version
   ```

   If corepack is unavailable, npm install -g yarn installs yarn globally instead.

6.3 Perform: Install the Dependencies

From inside the cloned project folder, one command reads package.json and installs everything:

```
yarn install
```

```
# or, for a project that uses npm:
# npm install
```

6.4 Verify

> **VERIFY**
>
> A node_modules folder now exists inside the project and contains many subfolders.
>
> yarn finished without red error output. Deprecation warnings are normal and harmless.
>
> A yarn.lock (or package-lock.json) file is present, recording exactly which versions were installed.

6.5 Troubleshooting Dependency Installs (Both Ecosystems)

Symptom	Likely cause	Fix
externally-managed-environment	Installing into system Python on a modern Linux	Create and activate a virtual environment first (Section 5.3), then install
ModuleNotFoundError when running the tool	The virtual environment is not active	Reactivate with source venv/bin/activate; confirm the (venv) prefix
pip: command not found	pip not installed	Install it: sudo dnf install -y python3-pip or the equivalent for your system
A single package fails to build (compiler errors)	A dependency needs system build tools or a header library	Install build tools: @development-tools on Fedora, build-essential on Ubuntu, then retry
yarn: command not found	yarn not enabled	Run corepack enable, or npm install -g yarn
Node install fails on a version error	Node too old for the project	Install the current LTS from nodejs.org, not the distribution package

Part 3: web-check

The first tool is **web-check** by Alicia Sykes, an all-in-one OSINT dashboard for analysing any website. You give it a domain and it reports the site's IP information, SSL certificate chain, DNS records, headers, cookies, open ports, server location, technology stack, and much more, all in one browser dashboard.

It is a Node.js application, so it follows the Node path from Part 2. Unlike the two Python tools, web-check runs as a small local web server that you open in a browser rather than a command-line program.

Property	Detail
Repository	https://github.com/lissy93/web-check
Language	TypeScript / JavaScript (Node.js)
What it does	Analyses a target website and presents an OSINT dashboard
Runtime needed	**Node.js 18.16.1 or newer**, plus yarn and git
Optional extras	chromium, traceroute, and dns on the host enable a few extra checks; missing ones are simply skipped
How you use it	Runs a local server, then you open a browser to it
License	MIT

7. Cloning and Installing web-check

7.1 Explain

The sequence is: clone the repo, install its Node dependencies with yarn, build the production version, then start the server. The build step compiles the application into an optimized form; the serve step launches both the dashboard and the API behind it.

7.2 Prerequisites

- Git installed (Part 1).
- Node.js 18.16.1 or newer, and yarn enabled (Part 2, Section 6).
- Around 1 GB of free disk space; Node projects have many dependencies.

7.3 Perform

1. Move into your tools folder:

```
cd ~/tools        # macOS / Linux
cd $HOME\tools    # Windows PowerShell
```

2. Clone the repository:

```
git clone https://github.com/lissy93/web-check.git
```

3. Step into the project folder:

```
cd web-check
```

4. Install the dependencies. This reads package.json and downloads everything into node_modules. It takes a few minutes:

```
yarn install
```

5. Build the application for production:

```
yarn build
```

6. Start the server, which serves both the dashboard and its API:

```
yarn serve
```

7. Open the address it prints, which is usually:

```
http://localhost:3000
```

[SCREENSHOT REQUIRED]

Capture: The terminal after yarn serve, alongside the web-check dashboard in a browser

Show: The server startup message with the localhost URL, and the web-check dashboard loaded at that address

Caption: Figure 7-1: web-check running locally and reachable in the browser

7.4 A Faster Alternative: Docker

If you already have Docker installed, web-check publishes a prebuilt image and you can skip cloning and building entirely:

```
docker run -p 3000:3000 lissy93/web-check
```

Then open http://localhost:3000 exactly as above. This is the quickest way to try the tool, though the from-source method above is the one that teaches the clone-and-install workflow this manual is about.

7.5 Configuration (Optional)

web-check runs with no configuration at all. Every core check works out of the box. Some checks that rely on external services return richer results if you supply free API keys, but none are required.

If you want them, copy the sample environment file and add keys:

```
cp .env.sample .env
# then edit .env and add any keys you have, for example:
# GOOGLE_CLOUD_API_KEY, REACT_APP_SHODAN_API_KEY, REACT_APP_WHO_API_KEY
```

WARNING

Keys whose names start with REACT_APP_ are used in the browser and can be seen by anyone inspecting network traffic. Only use keys scoped with minimum privileges there, and never put a sensitive, high-privilege key in a REACT_APP_ variable.

7.6 Using It, Briefly

With the dashboard open, type any domain into the box and submit. web-check runs its full battery of checks and fills the dashboard with cards: IP info, SSL chain, DNS records, headers, open ports, server location, and the rest.

> **WARNING**
> Only analyse sites you own or have permission to test. Most of what web-check does is passive lookups of public information, which is low risk. The port scan and a few active checks reach out to the target directly, so run those only against your own infrastructure or a site you are explicitly authorized to assess.

7.7 Verify

> **VERIFY**
> yarn install completed and a node_modules folder exists.
>
> yarn build completed without errors.
>
> yarn serve prints a running-server message with a localhost URL.
>
> The dashboard loads in your browser and returns results for a domain you enter.

7.8 Stopping and Restarting

- **Stop** the server by returning to its terminal and pressing Ctrl + C.
- **Restart** later by running cd ~/tools/web-check and then yarn serve again. You do not rebuild unless the code changed.
- **Update** to the latest version with git pull, then yarn install (in case dependencies changed) and yarn build again.

7.9 Troubleshooting

Symptom	Likely cause	Fix
yarn: command not found	yarn not enabled	corepack enable, or npm install -g yarn
Build fails on a Node version error	Node older than 18.16.1	Install the current LTS from nodejs.org and retry
Port 3000 is already in use	Another process holds the port	Stop the other process, or set a different port: PORT=3001 yarn serve
Some checks always show as skipped	Optional host tools missing	Install chromium, traceroute, or dns utilities if you want those specific checks
Blank dashboard after starting	Build step was skipped	Run yarn build before yarn serve

Part 4: GhostTrack

The second tool is **GhostTrack** by HunxByts, a small command-line OSINT utility that looks up publicly available information about an IP address, a phone number, or a username across social platforms. It is the simplest of the three to install, which makes it a good first Python tool to practice on.

Property	Detail
Repository	https://github.com/HunxByts/GhostTrack
Language	Python 3
What it does	Looks up public info for an IP, a phone number, or a username
Runtime needed	Python 3, plus git
Dependencies	Two packages: requests and phonenumbers
Entry point	GhostTR.py
Menu options	IP Tracker, Phone Tracker, Username Tracker

8. Cloning and Installing GhostTrack

8.1 Explain

GhostTrack is a plain Python script with two dependencies listed in a requirements.txt. The install is the classic Python sequence: clone, create a virtual environment, install the requirements, run the script. We use a virtual environment here even though the dependency list is short, because it is the correct habit and because modern Linux requires it.

8.2 Prerequisites

- Git installed (Part 1).
- Python 3 and pip (Part 2, Section 5).

8.3 Perform

1. Move into your tools folder:

```
cd ~/tools        # macOS / Linux
cd $HOME\tools    # Windows PowerShell
```

2. Clone the repository:

```
git clone https://github.com/HunxByts/GhostTrack.git
```

3. Step into the folder:

```
cd GhostTrack
```

4. Create and activate a virtual environment:

```
python3 -m venv venv
```

```
# macOS / Linux
source venv/bin/activate

# Windows PowerShell
venv\Scripts\Activate.ps1
```

Your prompt should now begin with (venv).

5. Install the two dependencies:

```
pip install -r requirements.txt
```

This installs requests (for making web requests) and phonenumbers (for parsing phone numbers).

6. Run the tool:

```
python3 GhostTR.py
```

The tool clears the screen and shows a banner followed by a numbered menu: IP Tracker, Phone Tracker, Username Tracker.

[SCREENSHOT REQUIRED]

Capture: The GhostTrack main menu after launch

Show: The GhostTrack banner and the numbered menu listing IP Tracker, Phone Tracker, and Username Tracker

Caption: Figure 8-1: GhostTrack running, showing its main menu

> **NOTE**
> GhostTrack was written with Linux and Termux in mind, and its README gives install notes for both. On plain Debian or Ubuntu the maintainer suggests sudo apt-get install git python3 first; on Termux, pkg install git python3. The clone-and-install steps above are the same everywhere once Git and Python are present.

8.4 Using It, Briefly

Pick a menu number and follow the prompts. The IP Tracker geolocates a public IP address. The Phone Tracker parses a number in international format (for example +14155550123) and reports the country, carrier, and line type. The Username Tracker checks a handle across a range of social platforms.

> **WARNING**
> This tool queries public data sources about whatever identifier you enter. Use it only for legitimate research, on identifiers you have a lawful reason to investigate. Looking up information to locate, harass, or surveil a private individual is not acceptable, and in many places it is illegal. Treat it as you would any investigative tool: purpose and authorization first.

8.5 Verify

> **VERIFY**
> The clone produced a GhostTrack folder containing GhostTR.py and requirements.txt.

> The virtual environment is active (the (venv) prefix is showing).
>
> pip install -r requirements.txt reported success for requests and phonenumbers.
>
> python3 GhostTR.py displays the banner and menu without a traceback.

8.6 Stopping and Restarting

- **Exit** the tool with its menu's quit option, or press Ctrl + C.
- **Leave** the virtual environment with deactivate.
- **Restart** later: cd ~/tools/GhostTrack, reactivate the environment, then python3 GhostTR.py.
- **Update** with git pull from inside the folder.

8.7 Troubleshooting

Symptom	Likely cause	Fix
ModuleNotFoundError: requests	Environment not active, or deps not installed	Reactivate the venv, then pip install -r requirements.txt
python3: command not found	Python 3 not installed or named python	Install Python 3, or try python GhostTR.py
Phone lookups return nothing	Number not in international format	Enter the number with its country code, for example +1…
externally-managed-environment on install	Installed outside a venv on modern Linux	Create and activate the venv first, then install
Banner shows but menu input is ignored	Terminal too small, or a paste issue	Enlarge the terminal and type the menu number directly

Part 5: theHarvester

The third tool is **theHarvester** by Christian Martorella and the laramies project, a long-standing OSINT reconnaissance tool used in the footprinting stage of a security assessment. Given a domain, it gathers subdomains, email addresses, hosts, IP addresses, and related data from a wide range of public sources such as search engines and certificate transparency logs.

theHarvester is the most actively maintained of the three, and its installation has modernized. The project now recommends a fast installer called **uv**, and the way you run it has changed from older tutorials. This part covers the current official method first, then the classic method as a fallback.

> **WARNING**
> The single most common mistake with theHarvester today comes from following old guides. There is **no longer a `theHarvester.py` script to run**. The command is now simply theHarvester (or uv run theHarvester). Any tutorial telling you to run python3 theHarvester.py is out of date and will fail.

Property	Detail
Repository	https://github.com/laramies/theHarvester
Language	Python 3
What it does	Passive reconnaissance: gathers subdomains, emails, hosts, and IPs from public sources
Runtime needed	**Python 3.12 or newer**, plus git
Recommended installer	uv (official), or pip in a virtual environment (classic)
Entry points	theHarvester and restfulHarvest. **Not** theHarvester.py.
API keys	Optional. Many sources work with no key; some richer sources need free keys in api-keys.yaml

9. Cloning and Installing theHarvester (Official uv Method)

9.1 Explain

uv is a modern Python project manager. It reads the project's pyproject.toml, creates an isolated environment automatically, and installs the exact locked dependency versions, all in one step. You do not create a virtual environment by hand; uv handles it. This is the method the maintainers now recommend.

9.2 Prerequisites

- Git installed (Part 1).
- Python 3.12 or newer. Check with python3 --version.

9.3 Perform

1. Install uv. The official one-line installer works on macOS and Linux:

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows PowerShell
powershell -ExecutionPolicy ByPass -c "irm
https://astral.sh/uv/install.ps1 | iex"
```

Close and reopen your terminal afterward so uv is on your PATH. Confirm with uv --version.

> **WARNING**
> That command pipes a script from the internet straight into your shell. That is what the vendor documents, but as a security practitioner you should recognize the pattern and treat it with appropriate care. If you prefer not to pipe-to-shell, uv is also installable through pipx install uv or your system package manager, and you can read the script first by opening the URL in a browser.

2. Move into your tools folder and clone the repository:

```
cd ~/tools
git clone https://github.com/laramies/theHarvester.git
cd theHarvester
```

3. Let uv create the environment and install all dependencies in one step:

```
uv sync
```

uv reads pyproject.toml, selects a compatible Python, builds an isolated environment in .venv, and installs the locked dependency versions.

4. Run theHarvester through uv to confirm it works:

```
uv run theHarvester -h
```

This prints the full help and options. uv run executes the command inside the environment uv just built, so you never have to activate anything manually.

[SCREENSHOT REQUIRED]
Capture: The theHarvester help output after uv run theHarvester -h
Show: The uv sync completion, then the theHarvester usage banner and its list of options
Caption: Figure 9-1: theHarvester installed via uv and printing its help

9.4 Perform: A First Passive Search

Run a harmless passive search against a domain intended for testing, using only sources that need no API key:

```
uv run theHarvester -d example.com -b crtsh,certspotter
```

-d is the target domain, and -b chooses the data sources; here two certificate-transparency sources that require no key. theHarvester queries them and prints any subdomains and hosts it finds.

9.5 Verify

> **VERIFY**
>
> uv --version prints a version number.
>
> The clone produced a theHarvester folder containing pyproject.toml.
>
> uv sync finished and created a .venv folder.
>
> uv run theHarvester -h prints the help banner with no traceback.
>
> A passive search against a test domain returns results or a clean "no results" message rather than an error.

10. Alternative: The Classic pip Method

10.1 Explain

If you cannot or would rather not use uv, theHarvester still installs the traditional way, with a hand-made virtual environment and pip. This mirrors exactly what you learned in Part 2 and what you did for GhostTrack, so it is worth seeing that the same pattern carries over.

10.2 Perform

1. Clone and enter the repository (skip if you already cloned it above):

```
cd ~/tools
git clone https://github.com/laramies/theHarvester.git
cd theHarvester
```

2. Create and activate a virtual environment:

```
python3 -m venv venv
source venv/bin/activate            # macOS / Linux
# venv\Scripts\Activate.ps1        # Windows PowerShell
```

3. Upgrade the packaging tools, then install the project from its own definition:

```
python3 -m pip install --upgrade pip setuptools wheel
python3 -m pip install .
```

   The dot means "install the project in this folder", which reads pyproject.toml and pulls in every dependency. This puts the theHarvester command inside your active environment.

4. Run it by its command name:

```
theHarvester -h
```

   Again: the command is theHarvester, not python3 theHarvester.py. The old launcher script no longer exists.

> **NOTE**
>
> On Kali Linux, theHarvester is also packaged directly: sudo apt update && sudo apt install theharvester. The packaged version can lag behind the repository, so a from-source install gives you the newest

sources and fixes. That trade-off, current features versus convenience, is worth pointing out to students.

10.3 Verify

VERIFY

The (venv) prefix is showing while you install and run.

theHarvester -h prints the help banner.

A passive search such as theHarvester -d example.com -b crtsh returns cleanly.

11. Configuration, Use, and Maintenance

11.1 API Keys (Optional)

Many of theHarvester's sources work with no configuration. Others, the ones that query commercial intelligence services, need a free or paid API key. On first run, theHarvester creates a config directory at ~/.theHarvester/, and you add keys to the api-keys.yaml file there. The project wiki lists which sources need keys and where to get them.

WARNING

Treat api-keys.yaml as a secret. It contains credentials. Do not commit it to a repository, do not paste it into a chat or ticket, and do not share your cloned folder with the file still populated.

11.2 Responsible Use

WARNING

theHarvester performs reconnaissance. Even though most of its sources are passive lookups of public data, gathering a target's subdomains, emails, and hosts is the opening move of an assessment and must be authorized. Run it only against domains you own or have explicit written permission to test. Unauthorized reconnaissance against third parties can violate computer misuse law even when every individual query hits a public source.

11.3 Stopping, Restarting, and Updating

- **uv method:** nothing to activate. From the folder, run uv run theHarvester ... whenever you need it. Update with git pull then uv sync.
- **pip method:** cd ~/tools/theHarvester, run source venv/bin/activate, then theHarvester .... Leave with deactivate. Update with git pull, reactivate, and pip install . again.

11.4 Troubleshooting

Symptom	Likely cause	Fix
python3 theHarvester.py: file not found	Following an outdated guide	Use the command theHarvester, or uv run theHarvester. There is no launcher script anymore.

Symptom	Likely cause	Fix
A Python version error at install	Python older than 3.12	Install Python 3.12+; with uv, uv can fetch a compatible Python automatically
uv: command not found	Terminal predates the uv install	Open a new terminal, or add uv to PATH as its installer instructed
A source returns "not supported" or empty	That source needs an API key, or a search engine is rate-limiting	Use key-free sources like crtsh,certspotter, or add the relevant key to api-keys.yaml
Screenshot feature errors on Linux	Playwright browser or system libraries missing	Follow the wiki's Playwright step, or simply avoid the screenshot option
externally-managed-environment	pip method without an active venv	Activate the virtual environment before installing

Appendices

Appendix A: The Whole Workflow on One Page

web-check (Node.js)

```
cd ~/tools
git clone https://github.com/lissy93/web-check.git
cd web-check
yarn install
yarn build
yarn serve
# open http://localhost:3000
```

GhostTrack (Python)

```
cd ~/tools
git clone https://github.com/HunxByts/GhostTrack.git
cd GhostTrack
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 GhostTR.py
```

theHarvester, official uv method (Python)

```
curl -LsSf https://astral.sh/uv/install.sh | sh   # once, then reopen terminal
cd ~/tools
git clone https://github.com/laramies/theHarvester.git
cd theHarvester
uv sync
uv run theHarvester -h
```

theHarvester, classic pip method (Python)

```
cd ~/tools/theHarvester
python3 -m venv venv
source venv/bin/activate
python3 -m pip install --upgrade pip setuptools wheel
python3 -m pip install .
theHarvester -h
```

Appendix B: Command Reference

Command	What it does
git --version	Check whether Git is installed
git clone <url>	Download a full copy of a repository
git pull	Update an already-cloned repository to the latest version

Command	What it does
cd <folder>	Move into a folder
ls -la / dir	List files in the current folder (macOS-Linux / Windows)
python3 -m venv venv	Create a Python virtual environment named venv
source venv/bin/activate	Activate the virtual environment (macOS / Linux)
venv\Scripts\Activate.ps1	Activate the virtual environment (Windows PowerShell)
deactivate	Leave the active virtual environment
pip install -r requirements.txt	Install Python dependencies listed in a file
pip install .	Install the project in the current folder and its dependencies
uv sync	Create an environment and install locked dependencies (uv)
uv run <command>	Run a command inside the uv-managed environment
corepack enable	Enable yarn (and other managers) that ship with Node.js
yarn install	Install Node.js dependencies from package.json
Ctrl + C	Stop a running program in the terminal

Appendix C: Glossary

Term	Definition
API key	A secret credential that identifies you to an external service, letting a tool query it on your behalf.
Build	The step that turns a project's source code into an optimized, runnable form.
Clone	A full local copy of a repository made with Git, still linked to the original for updates.
Dependency	A library a project needs in order to run, installed separately from the project itself.
Git	The version-control program on your computer that clones repositories and tracks changes.
GitHub	The website that hosts Git repositories, including all three tools in this manual.
Node.js	A runtime that lets JavaScript run outside a browser; web-check is a Node.js project.
npm / yarn	The two common installers for Node.js dependencies. web-check uses yarn.
OSINT	Open Source Intelligence: gathering information from publicly available sources.
pip	The standard installer for Python packages.

Term	Definition
Reconnaissance	The information-gathering stage of a security assessment. theHarvester is a recon tool.
Repository (repo)	A single project with all its files and its full change history.
requirements.txt	A file listing a Python project's dependencies, read by pip install -r.
Runtime	The language platform a program needs in order to run, such as Python 3 or Node.js.
uv	A fast, modern Python project manager that creates environments and installs dependencies in one step.
Virtual environment	An isolated Python workspace for one project, so its libraries cannot disturb others.

Appendix D: A Word on Safe and Lawful Use

All three tools in this manual gather information, and two of them can reach out to targets directly. That makes them genuinely useful for defenders and genuinely capable of causing harm if misused. Three rules keep you on the right side of the line.

- **Authorization first.** Only point these tools at systems, domains, or identifiers you own or have explicit permission to investigate. Written permission is best. "It's public data" is not by itself a defense when the activity is unauthorized reconnaissance against someone else.
- **Purpose matters.** These are instruments for security research and authorized assessment. Using them to locate, surveil, or harass a private person is both unethical and, in many jurisdictions, illegal.
- **Contain your lab.** When you are learning, run and test these tools against your own domains, deliberately vulnerable practice targets, or lab systems you control. That keeps your practice both safe and defensible.

> **NOTE**
> These same three tools are excellent for teaching the defensive side. Run web-check against your own site and you see exactly what an outsider can learn from your headers, DNS, and certificates, which tells you what to harden. Run theHarvester against your own domain and you see the footprint you are exposing. The most valuable lesson in this manual is not how to gather information about others, but how much others can gather about you, and what to do about it.